6901012610196 Chonpatchut Boonpalit Cpr.E S.2

เนื่องจากหนูไม่พบ error ในระหว่างเขียน หนูจึงเขียนเป็น error ที่อาจเกิดขึ้นแทนนะคะ

1. Lab2_01_1

โปรแกรมนี้เป็นการทดลองใช้คำสั่งพื้นฐานของ Processing เพื่อสร้าง Canvas และวาดเส้นตรงหนึ่งเส้น จุดประสงค์ของข้อนี้คือเพื่อให้เข้าใจระบบพิกัดของ Processing การทำงานของฟังก์ชัน `size()` และ `line()` รวมถึงลำดับการทำงานของโปรแกรมที่เริ่มทำงานจากบนลงล่าง

Pseudocode

สร้างพื้นที่สำหรับวาดภาพขนาด 500 × 500

วาดเส้นตรงจากจุด (0, 0) ไปยังจุด (250, 250)

อธิบายโค้ด

```
size(500, 500)
```

กำหนดขนาดของ Canvas ให้มีความกว้าง 500 pixels และความสูง 500 pixels โดยคำสั่งนี้ควรถูกเรียกก่อนคำสั่งวาดรูปทั้งหมด เพราะ Processing จะต้องสร้างพื้นที่สำหรับวาดภาพก่อน หากไม่กำหนดขนาด Canvas โปรแกรมจะใช้ค่าพื้นฐานแทน ซึ่งอาจไม่ตรงกับที่ต้องการ

```
line(0, 0, 250, 250)
```

ใช้สำหรับวาดเส้นตรงจากจุดเริ่มต้น `(0, 0)` ไปยังจุดสิ้นสุด `(250, 250)` โดยใน Processing จุด `(0, 0)` จะอยู่ที่มุมซ้ายบนของหน้าต่าง ไม่ใช่มุมซ้ายล่างเหมือนระบบพิกัดทางคณิตศาสตร์ ทำให้เส้นที่ได้เป็นเส้นทแยงจากมุมซ้ายบนมายังกึ่งกลางของหน้าจอ

Error ที่อาจเกิดขึ้น

หากเขียน

```
line(0, 0, 250)
```

จะเกิด **TypeError** เพราะฟังก์ชัน `line()` ต้องรับค่า 4 ตัว คือ `x1, y1, x2, y2`

หากเขียน

```
Line(0, 0, 250, 250)
```

จะเกิด **NameError** เนื่องจาก Python แยกตัวพิมพ์เล็กและตัวพิมพ์ใหญ่ คำสั่งที่ถูกต้องต้องเป็น `line()`

หากกำหนดค่าพิกัดเกินขนาด Canvas เช่น

`line(0, 0, 800, 800)`

โปรแกรมยังสามารถทำงานได้ แต่ปลายเส้นบางส่วนจะอยู่นอกพื้นที่แสดงผล ทำให้มองเห็นเพียงบางส่วนของเส้น

สิ่งที่ได้เรียนรู้

จากข้อนี้ทำให้เข้าใจว่าคำสั่งทุกคำสั่งใน Processing เป็นฟังก์ชัน และโปรแกรมจะทำงานตามลำดับจากบนลงล่าง นอกจากนี้ยังเข้าใจระบบพิกัดของ Processing และความสำคัญของการกำหนด Canvas ก่อนเริ่มวาดวัตถุ

2. Lab2_01_2

โปรแกรมนี้เป็นการทดลองใช้คำสั่ง `ellipse()` เพื่อวาดวงกลมและวงรีหลายรูปบน Canvas เดียวกัน จุดประสงค์ของข้อนี้คือเพื่อให้เข้าใจว่าฟังก์ชันเดียวกันสามารถนำมาใช้สร้างรูปร่างที่แตกต่างกันได้เพียงเปลี่ยนค่าพิกัดและขนาดที่ส่งเข้าไปเป็น argument นอกจากนี้ยังช่วยให้เข้าใจระบบพิกัดของ Processing มากขึ้น เพราะการกำหนดตำแหน่งที่ถูกต้องจะทำให้วัตถุปรากฏในตำแหน่งที่ต้องการ

Pseudocode

สร้าง Canvas ขนาด 500 × 500

วาดวงกลมตรงกลางหน้าจอ

วาดวงรีแนวตั้งบริเวณมุมซ้ายบน

วาดวงรีแนวนอนบริเวณมุมขวาบน

อธิบายโค้ด

size(500, 500)

กำหนดพื้นที่สำหรับวาดภาพให้มีขนาดกว้าง 500 pixels และสูง 500 pixels คำสั่งนี้ต้องอยู่ก่อนคำสั่งวาดวัตถุทั้งหมด เพื่อให้ Processing สร้างพื้นที่สำหรับแสดงผลก่อนเริ่มวาดรูป

ellipse(250, 250, 150, 150)

วาดวงกลมโดยใช้ฟังก์ชัน `ellipse()` เนื่องจากกำหนดความกว้างและความสูงเท่ากัน คือ 150 pixels จึงได้ผลลัพธ์เป็นวงกลม

- จุดกึ่งกลางอยู่ที่ (250, 250)
- ความกว้าง 150 pixels
- ความสูง 150 pixels

เนื่องจากตำแหน่ง (250, 250) อยู่กึ่งกลางของ Canvas ขนาด 500 × 500 วงกลมจึงอยู่ตรงกลางพอดี

ellipse(120, 120, 60, 100)

วาดวงรีอีกหนึ่งรูปที่บริเวณด้านซ้ายบนของหน้าจอ

- จุดกึ่งกลางอยู่ที่ (120, 120)
- ความกว้าง 60 pixels
- ความสูง 100 pixels

เนื่องจากความสูงมากกว่าความกว้าง จึงได้วงรีในแนวตั้ง

ellipse(380, 120, 100, 60)

วาดวงรีบริเวณด้านขวาบนของหน้าจอ

- จุดกึ่งกลางอยู่ที่ (380, 120)
- ความกว้าง 100 pixels
- ความสูง 60 pixels

เนื่องจากความกว้างมากกว่าความสูง จึงได้วงรีในแนวนอน

จากตัวอย่างนี้จะเห็นว่าฟังก์ชัน `ellipse()` เพียงฟังก์ชันเดียวสามารถสร้างได้ทั้งวงกลมและวงรี เพียงเปลี่ยนค่าความกว้างและความสูงที่ส่งเข้าไป

Error ที่อาจเกิดขึ้น

หากลืมใส่ argument ตัวใดตัวหนึ่ง เช่น

ellipse(250, 250, 150)

จะเกิด TypeError เนื่องจากฟังก์ชัน `ellipse()` ต้องรับค่า 4 ค่า ได้แก่ ตำแหน่งแกน x ตำแหน่งแกน y ความกว้าง และความสูง

หากพิมพ์ชื่อฟังก์ชันผิด เช่น

elipse(250, 250, 150, 150)

จะเกิด NameError เพราะ Python ไม่รู้จักชื่อฟังก์ชันดังกล่าว

หากกำหนดตำแหน่งเกินขนาด Canvas เช่น

ellipse(700, 700, 150, 150)

โปรแกรมจะยังสามารถทำงานได้ แต่รูปจะอยู่นอกพื้นที่แสดงผล ทำให้มองไม่เห็นวงกลม

หากกำหนดค่าความกว้างหรือความสูงเป็นศูนย์

ellipse(250, 250, 0, 150)

โปรแกรมจะทำงานได้ แต่รูปที่แสดงผลจะผิดจากที่ต้องการ เพราะความกว้างเป็นศูนย์

สิ่งที่ได้เรียนรู้

ข้อนี้ทำให้เข้าใจว่าฟังก์ชันเดียวสามารถสร้างผลลัพธ์ได้หลายรูปแบบจากการเปลี่ยนค่า parameter นอกจากนี้ยังเข้าใจการใช้ระบบพิกัดของ Processing มากขึ้น และเห็นความสำคัญของการกำหนดตำแหน่งและขนาดของวัตถุให้เหมาะสมก่อนเริ่มวาดภาพ

3. Lab2_01_3

โปรแกรมนี้เป็นการฝึกใช้ฟังก์ชัน `rect()` สำหรับวาดสี่เหลี่ยม พร้อมทั้งเรียนรู้การกำหนดตำแหน่งและขนาดของวัตถุบน Canvas จุดประสงค์ของข้อนี้คือเพื่อให้เข้าใจว่ารูปร่างแต่ละชนิดใน Processing มีฟังก์ชันเฉพาะของตัวเอง และสามารถนำมาประกอบกันเพื่อสร้างภาพที่ซับซ้อนมากขึ้นในอนาคต เช่น บ้าน หุ่นยนต์ หรือสิ่งของต่าง ๆ

นอกจากนี้ยังเป็นการฝึกสังเกตความแตกต่างระหว่างฟังก์ชัน `ellipse()` ที่ใช้จุดกึ่งกลางเป็นตำแหน่งอ้างอิง และ `rect()` ที่ใช้มุมซ้ายบนเป็นตำแหน่งเริ่มต้นของรูปสี่เหลี่ยม

Pseudocode

สร้าง Canvas ขนาด 500 × 500

วาดรูปสี่เหลี่ยมรูปที่หนึ่ง

วาดรูปสี่เหลี่ยมรูปที่สอง

วาดรูปสี่เหลี่ยมรูปที่สาม

ตรวจสอบตำแหน่งและขนาดของแต่ละรูป

อธิบายโค้ด

size(500, 500)

กำหนดพื้นที่สำหรับวาดภาพให้มีขนาดกว้าง 500 pixels และสูง 500 pixels เพื่อให้มีพื้นที่เพียงพอสำหรับวางรูปสี่เหลี่ยมทั้งหมด

rect(50, 50, 100, 150)

วาดรูปสี่เหลี่ยมผืนผ้า

- จุดเริ่มต้นอยู่ที่ (50, 50)
- ความกว้าง 100 pixels
- ความสูง 150 pixels

สำหรับคำสั่ง `rect()` ค่าพิกัดที่กำหนดไม่ใช่จุดกึ่งกลางของรูป แต่เป็นตำแหน่งของมุมซ้ายบน ดังนั้นหากต้องการขยับรูปไปทางขวา จะต้องเพิ่มค่าแกน x และหากต้องการขยับลงด้านล่าง จะต้องเพิ่มค่าแกน y

rect(200, 80, 120, 80)

วาดรูปสี่เหลี่ยมอีกหนึ่งรูป โดยกำหนดตำแหน่งและขนาดแตกต่างจากรูปแรก ทำให้เห็นว่าฟังก์ชันเดียวกันสามารถสร้างวัตถุหลายรูปได้ เพียงเปลี่ยนค่า argument ที่ส่งเข้าไป

rect(150, 250, 180, 120)

วาดรูปสี่เหลี่ยมรูปสุดท้ายบริเวณด้านล่างของ Canvas เพื่อฝึกการจัดวางวัตถุหลายชิ้นให้อยู่ในตำแหน่งที่เหมาะสมและไม่ทับซ้อนกัน

Error ที่อาจเกิดขึ้น

หากส่ง argument ไม่ครบ เช่น

rect(50, 50, 100)

จะเกิด TypeError เนื่องจากฟังก์ชัน `rect()` ต้องรับค่าอย่างน้อย 4 ค่า ได้แก่ ตำแหน่ง x ตำแหน่ง y ความกว้าง และความสูง

หากพิมพ์ชื่อฟังก์ชันผิด เช่น

rectangle(50, 50, 100, 150)

จะเกิด NameError เพราะไม่มีฟังก์ชันชื่อ `rectangle()` ใน Processing

หากกำหนดค่าความกว้างหรือความสูงเป็นค่าติดลบ โปรแกรมอาจแสดงผลผิดจากที่คาดไว้ หรือรูปอาจถูกวาดในทิศทางตรงกันข้าม ทำให้ตรวจสอบตำแหน่งได้ยาก

หากกำหนดตำแหน่งของรูปเกินขนาด Canvas รูปบางส่วนจะถูกตัดออกและมองเห็นไม่ครบ แม้ว่าโปรแกรมจะยังสามารถทำงานได้ตามปกติ

สิ่งที่ได้เรียนรู้

จากข้อนี้ทำให้เข้าใจหลักการทำงานของฟังก์ชัน `rect()` และเห็นความแตกต่างของระบบพิกัดเมื่อเทียบกับ `ellipse()` นอกจากนี้ยังได้ฝึกการวางตำแหน่งของวัตถุหลายชิ้นบน Canvas ซึ่งเป็นพื้นฐานสำคัญสำหรับการสร้างภาพที่ซับซ้อนขึ้นในข้อถัดไป และเป็นการเตรียมความพร้อมก่อนนำหลายรูปร่างมาประกอบกันเป็นภาพเดียวด้วยการใช้ฟังก์ชันที่ผู้ใช้สร้างขึ้น (user-defined function) ในลำดับต่อไป.

4. Lab2_01_4

โปรแกรมนี้เป็นการฝึกใช้ฟังก์ชัน `triangle()` สำหรับวาดรูปสามเหลี่ยม โดยจุดประสงค์หลักของข้อนี้คือการเรียนรู้ว่ารูปร่างแต่ละชนิดใน Processing มีจำนวน parameter ที่แตกต่างกัน และผู้เขียนโปรแกรมจำเป็นต้องเข้าใจว่าพิกัดแต่ละค่าหมายถึงจุดใดของรูป นอกจากนี้ยังช่วยฝึกการจินตนาการตำแหน่งของจุดบนระบบพิกัด เพื่อเตรียมความพร้อมสำหรับการสร้างภาพที่ประกอบจากหลายรูปทรงในงานที่ซับซ้อนขึ้น

Pseudocode

สร้าง Canvas ขนาด 500 × 500

กำหนดตำแหน่งของจุดยอดทั้งสามจุด

วาดรูปสามเหลี่ยมจากทั้งสามจุด

ตรวจสอบตำแหน่งของรูปสามเหลี่ยม

อธิบายโค้ด

size(500, 500)

กำหนดพื้นที่สำหรับวาดภาพให้มีขนาดกว้าง 500 pixels และสูง 500 pixels ซึ่งเป็นพื้นที่ที่ใช้แสดงผลของโปรแกรมทั้งหมด

triangle(250, 80, 120, 350, 380, 350)

ใช้ฟังก์ชัน `triangle()` เพื่อวาดรูปสามเหลี่ยม โดยฟังก์ชันนี้ต้องรับพิกัดทั้งหมด 6 ค่า ซึ่งแบ่งออกเป็น 3 จุด ได้แก่

จุดยอดที่ 1 คือ (250, 80)

จุดยอดที่ 2 คือ (120, 350)

จุดยอดที่ 3 คือ (380, 350)

Processing จะเชื่อมจุดทั้งสามเข้าด้วยกันโดยอัตโนมัติ ทำให้เกิดเป็นรูปสามเหลี่ยมหนึ่งรูป

เมื่อเปลี่ยนตำแหน่งของจุดใดจุดหนึ่ง รูปร่างของสามเหลี่ยมก็จะเปลี่ยนตามทันที ดังนั้นการกำหนดพิกัดอย่างถูกต้องจึงเป็นสิ่งสำคัญมาก

จากตัวอย่างนี้ จุดยอดด้านบนถูกกำหนดให้อยู่กึ่งกลางของหน้าจอ ส่วนอีกสองจุดอยู่ด้านล่าง ทำให้ได้รูปสามเหลี่ยมหน้าจั่วที่มีความสมดุล

Error ที่อาจเกิดขึ้น

หากส่งค่าไม่ครบทั้ง 6 ค่า เช่น

triangle(250, 80, 120, 350)

จะเกิด TypeError เนื่องจากจำนวน argument ไม่ครบตามที่ฟังก์ชันต้องการ

หากพิมพ์ชื่อฟังก์ชันผิด เช่น

triangel(250, 80, 120, 350, 380, 350)

จะเกิด NameError เพราะไม่มีฟังก์ชันชื่อดังกล่าว

หากกำหนดพิกัดของทั้งสามจุดให้อยู่ในตำแหน่งเดียวกัน หรืออยู่บนเส้นตรงเดียวกัน โปรแกรมยังสามารถทำงานได้ แต่จะไม่เห็นรูปสามเหลี่ยมตามที่ต้องการ เนื่องจากพื้นที่ของรูปมีค่าเป็นศูนย์

หากกำหนดพิกัดบางจุดอยู่นอก Canvas รูปสามเหลี่ยมบางส่วนจะถูกตัดออก แม้โปรแกรมจะยังสามารถทำงานได้ตามปกติ

สิ่งที่ได้เรียนรู้

ข้อนี้ทำให้เข้าใจว่าฟังก์ชันแต่ละตัวมีจำนวน parameter ที่แตกต่างกัน และผู้เขียนโปรแกรมต้องส่ง argument ให้ครบถ้วนจึงจะสามารถทำงานได้ นอกจากนี้ยังได้ฝึกการกำหนดตำแหน่งของจุดหลายจุดพร้อมกัน และเข้าใจว่าการเปลี่ยนค่าพิกัดเพียงเล็กน้อยสามารถเปลี่ยนรูปร่างของวัตถุได้อย่างชัดเจน ซึ่งเป็นพื้นฐานสำคัญของการสร้างภาพด้วยคอมพิวเตอร์กราฟิก และจะถูกนำไปใช้ต่อในการรวมหลายรูปทรงให้เป็นภาพเดียวในข้อถัดไป

5. Lab2_01_5

โปรแกรมนี้เป็นการฝึกใช้คำสั่งกำหนดสีและเส้นขอบของวัตถุ ได้แก่ `fill()` `stroke()` และ `strokeWeight()` ก่อนนำไปวาดรูปร่างต่าง ๆ จุดประสงค์ของข้อนี้คือเพื่อให้เข้าใจว่าคำสั่งบางคำสั่งไม่ได้สร้างวัตถุขึ้นมาโดยตรง แต่เป็นการกำหนด "สถานะการวาด (Drawing State)" ซึ่งจะส่งผลต่อวัตถุทุกชิ้นที่ถูกวาดหลังจากนั้น จนกว่าจะมีการเปลี่ยนค่าใหม่

แนวคิดนี้มีความสำคัญมาก เพราะเมื่อเริ่มสร้างภาพที่ซับซ้อน จะต้องควบคุมสีและความหนาของเส้นให้ถูกต้อง ไม่เช่นนั้นวัตถุทุกชิ้นอาจมีสีเดียวกันทั้งหมด ทำให้ภาพไม่ตรงกับที่ต้องการ

Pseudocode

สร้าง Canvas

กำหนดสีเส้นขอบ

กำหนดความหนาของเส้น

กำหนดสีสำหรับเติมวัตถุ

วาดรูปร่าง

เปลี่ยนสี

วาดรูปร่างถัดไป

อธิบายโค้ด

size(500, 500)

สร้างพื้นที่สำหรับวาดภาพ โดยกำหนดให้ Canvas มีขนาด 500 × 500 pixels

stroke(0)

กำหนดสีของเส้นขอบให้เป็นสีดำ โดยค่า 0 หมายถึงสีดำในระบบสีแบบ Grayscale

หลังจากเรียกใช้คำสั่งนี้ เส้นขอบของวัตถุทุกชิ้นที่วาดต่อจากนี้จะเป็นสีดำ จนกว่าจะมีการเรียก `stroke()` ใหม่

strokeWeight(3)

กำหนดความหนาของเส้นขอบให้มีค่า 3 pixels

หากไม่กำหนด Processing จะใช้ค่ามาตรฐาน ซึ่งอาจบางเกินไปสำหรับภาพบางประเภท การเพิ่มความหนาของเส้นช่วยให้รายละเอียดของรูปเห็นได้ชัดขึ้น

fill(255, 0, 0)

กำหนดสีสำหรับเติมวัตถุเป็นสีแดง โดยใช้ระบบสี RGB

Red = 255

Green = 0

Blue = 0

ค่าที่กำหนดจะถูกใช้กับวัตถุทุกชิ้นที่วาดหลังจากนี้

rect(100, 100, 150, 120)

วาดรูปสี่เหลี่ยมโดยใช้สีแดงที่กำหนดไว้ก่อนหน้า พร้อมเส้นขอบสีดำที่มีความหนา 3 pixels

fill(0, 255, 0)

เปลี่ยนสีเติมเป็นสีเขียว

คำสั่งนี้ไม่ได้เปลี่ยนสีของรูปก่อนหน้า แต่จะมีผลเฉพาะวัตถุที่วาดหลังจากคำสั่งนี้เท่านั้น

ellipse(330, 160, 120, 120)

วาดวงกลมโดยใช้สีเขียวแทนสีแดง เนื่องจากมีการเรียก `fill()` ใหม่ก่อนวาด

จากตัวอย่างนี้จะเห็นว่า การเปลี่ยนสีไม่จำเป็นต้องแก้ไขวัตถุเดิม เพียงกำหนดค่าใหม่ก่อนวาดวัตถุถัดไปก็เพียงพอแล้ว

Error ที่อาจเกิดขึ้น

หากลืมเรียก `fill()` ก่อนวาดวัตถุ โปรแกรมจะใช้สีล่าสุดที่ถูกกำหนดไว้ ทำให้สีของวัตถุไม่เป็นไปตามที่ต้องการ

หากกำหนดค่าสีเกินช่วง เช่น

fill(400, -20, 800)

Processing จะพยายามปรับค่าให้อยู่ในช่วงที่รองรับ ส่งผลให้สีที่ได้ไม่ตรงกับที่คาดไว้

หากลืมปิดวงเล็บ

fill(255, 0, 0

จะเกิด SyntaxError ทำให้โปรแกรมไม่สามารถรันได้

หากสะกดชื่อฟังก์ชันผิด เช่น

filll(255, 0, 0)

จะเกิด NameError เพราะไม่มีฟังก์ชันชื่อดังกล่าว

สิ่งที่ได้เรียนรู้

จากข้อนี้ทำให้เข้าใจว่า `fill()` `stroke()` และ `strokeWeight()` เป็นฟังก์ชันที่ใช้กำหนดคุณสมบัติของการวาด ไม่ได้สร้างวัตถุโดยตรง แต่จะมีผลกับวัตถุทุกชิ้นที่ถูกวาดหลังจากนั้น หลักการนี้ถูกนำไปใช้ตลอดการสร้างภาพในข้อถัด ๆ ไป เพราะช่วยให้สามารถควบคุมสีและลักษณะของวัตถุแต่ละส่วนได้อย่างเป็นระบบ และทำให้โค้ดอ่านง่ายขึ้นเมื่อต้องสร้างภาพที่มีองค์ประกอบจำนวนมาก

6. Lab2_01_6

โปรแกรมนี้เป็นการนำคำสั่งวาดรูปร่างพื้นฐานที่ได้เรียนมาก่อนหน้านี้มาประกอบกันเป็นภาพเดียว จุดประสงค์ของข้อนี้คือเพื่อฝึกการวางแผนก่อนเขียนโปรแกรม การแบ่งวัตถุออกเป็นส่วนย่อย และการจัดลำดับการวาดให้ถูกต้อง เนื่องจากใน Processing วัตถุที่ถูกวาดภายหลังจะอยู่ด้านหน้าของวัตถุที่วาดก่อน หากเรียงลำดับไม่ถูกต้อง อาจทำให้บางส่วนของภาพถูกบังจนไม่เห็น

ข้อนี้ถือเป็นพื้นฐานสำคัญก่อนเริ่มเรียนเรื่อง User-defined Function เพราะเมื่อภาพมีองค์ประกอบจำนวนมาก การแบ่งโค้ดออกเป็นฟังก์ชันจะช่วยให้โปรแกรมเป็นระเบียบมากขึ้น และสามารถกลับมาแก้ไขเฉพาะส่วนได้ง่าย

Pseudocode

สร้าง Canvas

กำหนดสีและความหนาของเส้น

วาดส่วนที่เป็นพื้นหลัง

วาดส่วนหลักของรูป

วาดรายละเอียดเพิ่มเติม

ตรวจสอบลำดับการวาดของทุกองค์ประกอบ

อธิบายโค้ด

size(1000, 800)

กำหนดพื้นที่สำหรับวาดภาพให้มีขนาดกว้าง 1000 pixels และสูง 800 pixels เพื่อรองรับองค์ประกอบทั้งหมดของภาพ โดยเลือกใช้พื้นที่ที่มีขนาดค่อนข้างใหญ่เพื่อให้สามารถจัดวางวัตถุต่าง ๆ ได้สะดวก

strokeWeight(4)

กำหนดความหนาของเส้นขอบให้เป็น 4 pixels เพื่อให้รายละเอียดของรูปเห็นได้ชัดเจนขึ้น หากใช้ค่าที่เล็กเกินไป เส้นอาจดูบางจนมองเห็นรายละเอียดได้ยาก

fill(...)

กำหนดสีของวัตถุแต่ละส่วน เช่น ส่วนหัว ใบหน้า เสื้อผ้า หรือส่วนประกอบอื่น ๆ ก่อนเรียกคำสั่งวาดรูปร่างจริง การเปลี่ยนสีจะทำก่อนวาดวัตถุแต่ละชิ้น เพื่อให้แต่ละส่วนมีสีแตกต่างกันอย่างถูกต้อง

ellipse(...)

ใช้สร้างวัตถุที่เป็นลักษณะวงกลมหรือวงรี เช่น ศีรษะ ดวงตา หรือส่วนประกอบที่มีลักษณะโค้ง การเลือกใช้ `ellipse()` ช่วยลดจำนวนคำสั่งเมื่อเทียบกับการวาดจากเส้นหลายเส้น

rect(...)

ใช้สร้างส่วนที่มีลักษณะเป็นสี่เหลี่ยม เช่น ลำตัว หรือวัตถุที่มีรูปทรงเหลี่ยม ทำให้ภาพดูมีโครงสร้างที่ชัดเจนขึ้น

triangle(...)

ใช้สร้างส่วนที่มีลักษณะเป็นสามเหลี่ยม เช่น หู หลังคา หรือส่วนตกแต่งต่าง ๆ โดยอาศัยการกำหนดจุดยอดทั้งสามตำแหน่ง

line(...)

ใช้สร้างรายละเอียดเพิ่มเติม เช่น เส้นแบ่ง เส้นตกแต่ง หรือองค์ประกอบเล็ก ๆ ที่ไม่สามารถใช้รูปร่างพื้นฐานอื่นแทนได้

จากการรวมคำสั่งทั้งหมดเข้าด้วยกัน จะได้ภาพที่สมบูรณ์กว่าข้อก่อน ๆ และเริ่มเห็นแนวคิดของการสร้างภาพจากวัตถุหลายชิ้น (Composition) ซึ่งเป็นพื้นฐานของงานกราฟิกและเกม

Error ที่อาจเกิดขึ้น

หากวาดวัตถุผิดลำดับ เช่น วาดใบหน้าก่อน แล้วจึงวาดศีรษะในภายหลัง ศีรษะจะทับใบหน้าทั้งหมด ทำให้รายละเอียดบางส่วนหายไป แม้โปรแกรมจะไม่เกิด Error แต่ผลลัพธ์จะไม่ถูกต้อง ซึ่งจัดเป็น **Logical Error**

หากลืมเปลี่ยนสีด้วย `fill()` วัตถุหลายชิ้นจะมีสีเดียวกัน ทำให้ภาพขาดรายละเอียด แม้โปรแกรมจะยังทำงานได้ตามปกติ

หากกำหนดตำแหน่งของวัตถุผิดเพียงเล็กน้อย เช่น ขยับตาไปทางซ้ายหรือขวามากเกินไป ภาพที่ได้อาจดูผิดสัดส่วน ดังนั้นการกำหนดค่าพิกัดจึงต้องอาศัยการทดลองและปรับแต่งหลายครั้ง

หากสะกดชื่อฟังก์ชันผิด เช่น `elipse()` หรือ `traingle()` จะเกิด **NameError** ทันที เพราะ Python ไม่รู้จักชื่อฟังก์ชันดังกล่าว

หากลืมเครื่องหมายวงเล็บหรือเครื่องหมายจุลภาคระหว่าง argument จะเกิด **SyntaxError** ทำให้โปรแกรมไม่สามารถรันได้

สิ่งที่ได้เรียนรู้

จากข้อนี้ทำให้เข้าใจว่าการสร้างภาพหนึ่งภาพไม่ได้เกิดจากคำสั่งเพียงคำสั่งเดียว แต่เป็นการนำรูปร่างพื้นฐานหลายชนิดมาประกอบกันอย่างเป็นลำดับ นอกจากนี้ยังได้ฝึกการคิดเป็นขั้นตอน (Computational Thinking) โดยเริ่มจากการแบ่งภาพออกเป็นส่วนย่อย แล้วค่อยสร้างแต่ละส่วนทีละลำดับ ซึ่งช่วยลดความซับซ้อนของโปรแกรมและทำให้สามารถตรวจสอบข้อผิดพลาดได้ง่ายขึ้น

ข้อนี้ยังเป็นการเตรียมความพร้อมก่อนเข้าสู่การใช้ **User-defined Function** ในข้อถัดไป เนื่องจากเมื่อจำนวนคำสั่งมีมากขึ้น การรวมทุกอย่างไว้ในไฟล์เดียวจะทำให้โค้ดยาวและอ่านยาก การแบ่งโค้ดออกเป็นฟังก์ชันจะช่วยให้โปรแกรมเป็นระเบียบ สามารถเรียกใช้ซ้ำได้ และสะดวกต่อการแก้ไขในอนาคต ซึ่งเป็นหนึ่งในจุดประสงค์สำคัญของ Lab นี้ตามที่อาจารย์ต้องการให้เรียนรู้

7. Lab02_02

โปรแกรมนี้เป็นการวาดคำว่า **LINE** โดยใช้คำสั่ง `line()` หลายเส้นมาประกอบกันเป็นตัวอักษร L, I, N และ E จุดประสงค์ของข้อนี้คือเพื่อฝึกสร้างและเรียกใช้ **User-defined Function** รวมถึงเรียนรู้การส่งค่า **parameter** และ **argument** ให้กับฟังก์ชัน เพื่อให้สามารถกำหนดตำแหน่งและขนาดของตัวอักษรแต่ละตัวได้อย่างยืดหยุ่น

แทนที่จะเขียนคำสั่ง `line()` ทั้งหมดไว้ใน `setup()` โปรแกรมได้แบ่งการวาดตัวอักษรแต่ละตัวออกเป็นฟังก์ชัน ได้แก่ `draw_L()`, `draw_I()`, `draw_N()` และ `draw_E()` ทำให้โค้ดอ่านง่าย แก้ไขง่าย และสามารถเรียกใช้ซ้ำได้ในตำแหน่งอื่นโดยไม่ต้องเขียนโค้ดใหม่ทั้งหมด

Pseudocode

สร้างฟังก์ชันสำหรับวาดตัวอักษร L

สร้างฟังก์ชันสำหรับวาดตัวอักษร I

สร้างฟังก์ชันสำหรับวาดตัวอักษร N

สร้างฟังก์ชันสำหรับวาดตัวอักษร E

สร้าง Canvas ขนาด 600 × 300 pixels

กำหนดระยะห่างระหว่างตัวอักษร

เรียกใช้ฟังก์ชันวาดตัวอักษร L

เรียกใช้ฟังก์ชันวาดตัวอักษร I

เรียกใช้ฟังก์ชันวาดตัวอักษร N

เรียกใช้ฟังก์ชันวาดตัวอักษร E

แสดงผลเป็นคำว่า LINE

อธิบายโค้ด

def draw_L(x, y, w, h):

สร้างฟังก์ชันชื่อ `draw_L()` สำหรับวาดตัวอักษร L โดยรับค่า 4 ค่า ได้แก่

- x คือ ตำแหน่งเริ่มต้นในแนวนอน
- y คือ ตำแหน่งเริ่มต้นในแนวตั้ง
- w คือ ความกว้างของตัวอักษร
- h คือ ความสูงของตัวอักษร

การใช้ parameter ทำให้สามารถนำฟังก์ชันเดิมไปวาดตัว L ได้หลายตำแหน่งหรือหลายขนาด โดยไม่ต้องแก้ไขคำสั่งภายในฟังก์ชัน

line(x, y, x, y+h)

วาดเส้นแนวตั้งด้านซ้ายของตัวอักษร L

จุดเริ่มต้นอยู่ที่ `(x, y)`

จุดสิ้นสุดอยู่ที่ `(x, y+h)`

เนื่องจากค่า x ของทั้งสองจุดเท่ากัน เส้นที่ได้จึงเป็นเส้นแนวตั้ง

line(x, y+h, x+w, y+h)

วาดเส้นแนวนอนด้านล่างของตัวอักษร L

จุดเริ่มต้นอยู่ที่ `(x, y+h)`

จุดสิ้นสุดอยู่ที่ `(x+w, y+h)`

เมื่อรวมเส้นแนวตั้งและเส้นแนวนอนเข้าด้วยกัน จะได้ตัวอักษร L

---

def draw_I(x, y, w, h):

สร้างฟังก์ชันสำหรับวาดตัวอักษร I โดยรับ parameter 4 ตัวเช่นเดียวกับฟังก์ชันก่อนหน้า ทำให้ทุกฟังก์ชันมีรูปแบบการใช้งานเหมือนกัน

line(x, y, x+w, y)

วาดเส้นแนวนอนด้านบนของตัวอักษร I

line(x+w/2, y, x+w/2, y+h)

วาดเส้นแนวตั้งตรงกลางของตัวอักษร

ตำแหน่ง x+w/2 ใช้คำนวณกึ่งกลางของตัวอักษร ทำให้เส้นอยู่ตรงกลางพอดี

line(x, y+h, x+w, y+h)

วาดเส้นแนวนอนด้านล่างของตัวอักษร I

เมื่อรวมทั้งสามเส้นเข้าด้วยกัน จะได้ตัวอักษร I

---

def draw_N(x, y, w, h):

สร้างฟังก์ชันสำหรับวาดตัวอักษร N

line(x, y, x, y+h)

วาดเส้นแนวตั้งด้านซ้ายของตัวอักษร

line(x, y, x+w, y+h)

วาดเส้นทแยงจากมุมซ้ายบนไปยังมุมขวาล่าง

เส้นนี้เป็นส่วนสำคัญที่ทำให้ตัวอักษรมีลักษณะเป็น N

line(x+w, y, x+w, y+h)

วาดเส้นแนวตั้งด้านขวาของตัวอักษร

เมื่อรวมทั้งสามเส้น จะได้ตัวอักษร N

---

def draw_E(x, y, w, h):

สร้างฟังก์ชันสำหรับวาดตัวอักษร E

line(x, y, x+w, y)

วาดเส้นแนวนอนด้านบน

line(x, y+h/2, x+w, y+h/2)

วาดเส้นแนวนอนตรงกลางของตัวอักษร โดยใช้ y+h/2 เพื่อหาตำแหน่งกึ่งกลางในแนวตั้ง

line(x, y+h, x+w, y+h)

วาดเส้นแนวนอนด้านล่าง

line(x, y, x, y+h)

วาดเส้นแนวตั้งด้านซ้าย

เมื่อรวมเส้นทั้งหมดเข้าด้วยกัน จะได้ตัวอักษร E

---

def setup():

เป็นฟังก์ชันหลักของ Processing ซึ่งจะทำงานเพียงครั้งเดียวเมื่อเริ่มต้นโปรแกรม

size(600, 300)

กำหนดขนาด Canvas ให้กว้าง 600 pixels และสูง 300 pixels เพื่อให้มีพื้นที่เพียงพอสำหรับแสดงตัวอักษรทั้งหมด

gap = 80

สร้างตัวแปรชื่อ `gap` เพื่อใช้เก็บระยะห่างระหว่างตัวอักษร

การใช้ตัวแปรแทนตัวเลขช่วยให้สามารถปรับระยะห่างของตัวอักษรทั้งหมดได้ง่าย เพียงแก้ไขค่า `gap` เพียงตำแหน่งเดียว

draw_L(50, 80, 60, 120)

เรียกใช้ฟังก์ชัน `draw_L()` เพื่อวาดตัวอักษร L

ค่า 50, 80, 60 และ 120 เป็น **argument** ที่ถูกส่งไปยัง parameter คือ x, y, w และ h ตามลำดับ

draw_I(50 + 60 + gap, 70, 60, 120)

เรียกใช้ฟังก์ชัน `draw_I()` โดยคำนวณตำแหน่ง x จาก

50 + 60 + gap

เมื่อ gap มีค่า 80 จะได้ตำแหน่ง x เท่ากับ 190

draw_N(50 + (60 + gap) * 2, 70, 60, 120)

เรียกใช้ฟังก์ชัน `draw_N()`

ตำแหน่ง x คำนวณได้เป็น

50 + (60 + 80) × 2

เท่ากับ 330

draw_E(50 + (60 + gap) * 3, 70, 60, 120)

เรียกใช้ฟังก์ชัน `draw_E()`

ตำแหน่ง x คำนวณได้เป็น

50 + (60 + 80) × 3

เท่ากับ 470

การคำนวณตำแหน่งในลักษณะนี้ช่วยให้ตัวอักษรทุกตัวมีระยะห่างเท่ากัน และหากต้องการเปลี่ยนระยะห่างก็สามารถแก้ไขที่ตัวแปร `gap` เพียงจุดเดียว

Error ที่อาจเกิดขึ้น

หากลืมเยื้องคำสั่งภายในฟังก์ชัน เช่น

```
def draw_L(x, y, w, h):
line(x, y, x, y+h)
```

จะเกิด **IndentationError** เพราะ Python ใช้การเยื้องบรรทัดเพื่อกำหนดขอบเขตของฟังก์ชัน

หากเรียกใช้ฟังก์ชันโดยส่ง argument ไม่ครบ เช่น

```
draw_L(50, 80, 60)
```

จะเกิด **TypeError** เพราะฟังก์ชัน `draw_L()` ต้องรับค่า 4 ค่า

หากพิมพ์ชื่อฟังก์ชันผิด เช่น

```
draw_l(50, 80, 60, 120)
```

จะเกิด **NameError** เนื่องจาก Python แยกตัวพิมพ์เล็กและตัวพิมพ์ใหญ่

หากลืมใส่เครื่องหมาย `:` หลังการประกาศฟังก์ชัน เช่น

```
def draw_L(x, y, w, h)
```

จะเกิด **SyntaxError**

หากกำหนดค่า `gap` น้อยเกินไป ตัวอักษรแต่ละตัวอาจอยู่ชิดหรือซ้อนกัน ทำให้อ่านคำว่า LINE ได้ยาก

หากกำหนดค่า `gap` มากเกินไป ตัวอักษร E อาจถูกวาดออกนอก Canvas แม้โปรแกรมจะยังทำงานได้ แต่ผลลัพธ์จะไม่ตรงตามที่ต้องการ

สิ่งที่ได้เรียนรู้

ข้อนี้ทำให้เข้าใจการสร้างและเรียกใช้ **User-defined Function** มากขึ้น โดยแต่ละฟังก์ชันรับผิดชอบการวาดตัวอักษรเพียงหนึ่งตัว ทำให้โค้ดเป็นระเบียบและลดการเขียนคำสั่งซ้ำ นอกจากนี้ยังได้เรียนรู้การส่งค่า **parameter** และ **argument** เพื่อกำหนดตำแหน่งและขนาดของตัวอักษร รวมถึงเห็นประโยชน์ของการใช้ตัวแปรในการคำนวณตำแหน่ง เช่น `gap` ที่ช่วยให้สามารถปรับระยะห่างของตัวอักษรทั้งหมดได้อย่างสะดวกและทำให้โปรแกรมมีความยืดหยุ่นมากขึ้น

8. Lab1_03

โปรแกรมนี้เป็นการสร้าง Animation โดยใช้ฟังก์ชัน `draw()` ของ Processing ทำให้ภาพถูกวาดใหม่อย่างต่อเนื่องประมาณ 60 ครั้งต่อวินาที ส่งผลให้วัตถุบนหน้าจอดูเหมือนกำลังเคลื่อนที่จริง จุดประสงค์ของข้อนี้คือเพื่อให้นักศึกษาเข้าใจหลักการทำงานของแอนิเมชัน การแบ่งหน้าที่ของฟังก์ชัน การใช้ตัวแปรเพื่อเก็บสถานะของวัตถุ และการใช้คำสั่งเงื่อนไขในการควบคุมทิศทางการเคลื่อนที่

ต่างจากข้อก่อน ๆ ที่วาดภาพเพียงครั้งเดียว โปรแกรมข้อนี้ต้องคำนวณตำแหน่งใหม่ของวัตถุทุกเฟรม ทำให้เริ่มเห็นแนวคิดของการเขียนโปรแกรมแบบ Real-time ซึ่งถูกใช้ในเกม โปรแกรมจำลอง และงานด้าน Computer Vision

จุดประสงค์ของข้อนี้

- เข้าใจหลักการสร้าง Animation
- เข้าใจการทำงานของฟังก์ชัน `draw()`
- ฝึกใช้ Global Variable เพื่อเก็บสถานะของโปรแกรม
- ฝึกใช้ Conditional Statement ควบคุมการเคลื่อนที่
- เรียนรู้การแบ่งโปรแกรมออกเป็นหลายฟังก์ชัน

Pseudocode

สร้างตัวแปรเก็บตำแหน่งเริ่มต้นของตัวละคร

สร้าง Canvas

กำหนดค่าพื้นฐาน

ทำงานภายใน draw()

ล้างหน้าจอ

วาดตัวละคร

คำนวณตำแหน่งใหม่

ตรวจสอบว่าชนขอบหรือไม่

หากชนขอบให้เปลี่ยนทิศทาง

แสดงผลเฟรมถัดไป

อธิบายโค้ด

global x

global y

ตัวแปรทั้งสองใช้เก็บตำแหน่งปัจจุบันของตัวละครบนหน้าจอ

เหตุผลที่ต้องประกาศเป็น Global Variable เพราะค่าของตำแหน่งจะต้องถูกใช้งานร่วมกันระหว่างหลายฟังก์ชัน เช่น `setup()`, `draw()` และฟังก์ชันสำหรับวาดตัวละคร

หากประกาศเป็น Local Variable ทุกครั้งที่ `draw()` ทำงาน ตัวแปรจะถูกสร้างใหม่ ทำให้ตำแหน่งกลับไปเริ่มต้นทุกเฟรม ส่งผลให้ตัวละครไม่สามารถเคลื่อนที่ได้

ดังนั้น Global Variable จึงเหมาะสำหรับข้อมูลที่ต้องเก็บสถานะต่อเนื่องตลอดอายุของโปรแกรม

---

setup()

ฟังก์ชันนี้จะทำงานเพียงครั้งเดียวเมื่อเริ่มรันโปรแกรม

ภายในมักใช้สำหรับ

กำหนดขนาด Canvas

กำหนด Frame Rate

กำหนดตำแหน่งเริ่มต้น

โหลดรูปภาพหรือ Font

เนื่องจากข้อมูลเหล่านี้ไม่จำเป็นต้องคำนวณใหม่ทุกเฟรม จึงเหมาะกับการอยู่ใน `setup()`

---

draw()

ฟังก์ชันนี้จะถูกเรียกซ้ำประมาณ 60 ครั้งต่อวินาที

ทุกครั้งที่ `draw()` ทำงาน โปรแกรมจะ

ล้างหน้าจอ

คำนวณตำแหน่งใหม่

วาดตัวละครใหม่

แสดงผลบนหน้าจอ

กระบวนการนี้เกิดขึ้นอย่างรวดเร็ว จึงทำให้ผู้ใช้มองเห็นเป็นการเคลื่อนที่อย่างต่อเนื่อง

---

background(...)

ล้างภาพของเฟรมก่อนหน้า

หากไม่เรียก `background()` ทุกเฟรม ตัวละครจะทิ้งรอยไว้ตลอดเส้นทางที่เคลื่อนที่ ทำให้ภาพดูไม่ถูกต้อง

---

draw_character()

เรียกฟังก์ชันสำหรับวาดตัวละคร

การแยกออกเป็นฟังก์ชันช่วยให้ `draw()` มีหน้าที่เพียงควบคุมลำดับการทำงาน ส่วนรายละเอียดของตัวละครจะถูกจัดการในอีกฟังก์ชันหนึ่ง

แนวคิดนี้ทำให้โค้ดอ่านง่ายและแก้ไขสะดวก

---

x += speed_x

นำค่าความเร็วไปบวกกับตำแหน่งเดิม

หากความเร็วเป็นบวก ตัวละครจะเคลื่อนไปทางขวา

หากความเร็วเป็นลบ ตัวละครจะเคลื่อนไปทางซ้าย

หลักการเดียวกันใช้กับแกน y

---

if x >= width:

ตรวจสอบว่าตัวละครเดินชนขอบด้านขวาหรือไม่

หากเงื่อนไขเป็นจริง จะเข้าสู่คำสั่งภายใน if

---

speed_x *= -1

เปลี่ยนทิศทางการเคลื่อนที่

เดิม

speed_x = 5

จะกลายเป็น

speed_x = -5

ทำให้ตัวละครเริ่มเคลื่อนกลับทางซ้าย

หลักการเดียวกันใช้กับการชนขอบด้านซ้าย ด้านบน และด้านล่าง

ประโยชน์ของ Global Variable

ข้อนี้เป็นตัวอย่างที่เห็นได้ชัดที่สุดของการใช้ Global Variable

ตัวแปรตำแหน่งของตัวละครจำเป็นต้องถูกใช้งานหลายครั้งตลอดอายุของโปรแกรม

หากประกาศเป็น Local Variable ภายใน `draw()`

ตำแหน่งจะกลับมาเป็นค่าเดิมทุกเฟรม

Animation จะไม่เกิดขึ้น

Global Variable จึงช่วยให้ข้อมูลสามารถถูกแก้ไขและนำกลับมาใช้ต่อได้ทุกครั้งที่ `draw()` ทำงาน

อย่างไรก็ตาม การใช้ Global มากเกินไปก็มีข้อเสีย เพราะทุกฟังก์ชันสามารถแก้ไขค่าเดียวกันได้ ทำให้ตรวจสอบข้อผิดพลาดได้ยาก ดังนั้นควรใช้เฉพาะข้อมูลที่จำเป็นต้องแชร์จริง ๆ เช่น ตำแหน่ง ความเร็ว หรือคะแนนของเกม

การใช้ Conditional Statement

โปรแกรมนี้ใช้คำสั่ง if เพื่อตรวจสอบสถานะของตัวละคร

ตัวอย่างเช่น

หากตัวละครชนขอบด้านขวา

ให้เปลี่ยนความเร็วเป็นค่าติดลบ

หากชนขอบด้านซ้าย

ให้เปลี่ยนความเร็วกลับเป็นค่าบวก

การใช้เงื่อนไขทำให้โปรแกรมสามารถตอบสนองต่อเหตุการณ์ที่เกิดขึ้นได้โดยอัตโนมัติ ซึ่งเป็นพื้นฐานของเกม โปรแกรมจำลอง และระบบโต้ตอบกับผู้ใช้

Error ที่พบหรืออาจเกิดขึ้น

1. UnboundLocalError

หากลืมประกาศ

global x

ภายในฟังก์ชันที่มีการแก้ไขค่า เช่น

x += speed

Python จะเข้าใจว่า x เป็น Local Variable

แต่ยังไม่ได้ถูกสร้าง

จึงเกิด UnboundLocalError

วิธีแก้คือประกาศ

global x

ก่อนใช้งาน

---

2. NameError

หากเรียกใช้ตัวแปรที่ยังไม่ถูกสร้าง เช่น

target_x

โดยยังไม่ได้กำหนดค่า

Python จะรายงานว่าไม่พบตัวแปรดังกล่าว

---

3. IndentationError

หากลืมเยื้องคำสั่งภายใน if

Python จะไม่สามารถทราบได้ว่าคำสั่งใดอยู่ภายใต้เงื่อนไข

---

4. Logical Error

หากลืมเปลี่ยนเครื่องหมายของความเร็วหลังชนขอบ

ตัวละครจะพยายามเคลื่อนที่ออกนอกหน้าจอเรื่อย ๆ

แม้โปรแกรมจะไม่เกิด Error แต่ผลลัพธ์จะไม่ถูกต้อง

---

5. Animation Trail

หากลืมเรียก

background()

ภายใน draw()

ภาพของเฟรมก่อนหน้าจะไม่ถูกลบ

ตัวละครจะทิ้งรอยไว้ตลอดเส้นทางการเคลื่อนที่

แม้ไม่ใช่ Error ของภาษา แต่ถือเป็นข้อผิดพลาดเชิงตรรกะ (Logical Error)

สิ่งที่ได้เรียนรู้

ข้อนี้เป็นการรวมองค์ความรู้เกือบทั้งหมดของ Lab เข้าด้วยกัน ทั้งการสร้างฟังก์ชัน การเรียกใช้ฟังก์ชัน การส่งค่าให้ฟังก์ชัน การใช้ตัวแปรแบบ Global และการใช้คำสั่งเงื่อนไขเพื่อควบคุมการทำงานของโปรแกรม นอกจากนี้ยังได้เรียนรู้หลักการสร้าง Animation ที่อาศัยการวาดภาพซ้ำอย่างต่อเนื่องภายใน `draw()` ซึ่งเป็นพื้นฐานสำคัญของการพัฒนาเกม โปรแกรมกราฟิก และงานด้าน Computer Vision ที่ต้องแสดงผลแบบ Real-time การทำ Lab ข้อนี้ทำให้เห็นภาพชัดเจนว่าการแบ่งโปรแกรมออกเป็นฟังก์ชันและการเลือกใช้ตัวแปรอย่างเหมาะสมช่วยให้โปรแกรมมีโครงสร้างที่ดี อ่านง่าย และต่อยอดได้ในอนาคต

9. say_hello

โปรแกรมนี้เป็นการฝึกใช้ **Conditional Statement (if-elif-else)** ร่วมกับ **User-defined Function** โดยผู้ใช้จะป้อนตัวเลขผ่านคีย์บอร์ด จากนั้นโปรแกรมจะเลือกแสดงข้อความทักทายในภาษาที่แตกต่างกันตามค่าที่ผู้ใช้เลือก หากผู้ใช้ป้อนค่าที่ไม่ตรงกับเงื่อนไข โปรแกรมจะแจ้งเตือนว่าค่าไม่ถูกต้อง

จุดประสงค์ของข้อนี้คือเพื่อให้เข้าใจหลักการทำงานของคำสั่งเงื่อนไข การเรียกใช้ฟังก์ชัน และการส่ง Argument เข้าไปยังฟังก์ชัน ซึ่งเป็นพื้นฐานสำคัญของการเขียนโปรแกรมที่ต้องมีการตัดสินใจตามข้อมูลที่ได้รับ

จุดประสงค์ของข้อนี้

- ฝึกสร้าง User-defined Function
- ฝึกส่ง Argument เข้าไปใน Function
- ฝึกใช้ Parameter ภายใน Function
- ฝึกใช้คำสั่ง if, elif และ else
- ฝึกการรับข้อมูลจากผู้ใช้ด้วย input()

Pseudocode

สร้างฟังก์ชัน say_hello(choice)

ตรวจสอบค่าของ choice

ถ้าเป็น 1 ให้แสดง Hello

ถ้าเป็น 2 ให้แสดง Sawasdee

ถ้าเป็น 3 ให้แสดง Bonjour

ถ้าไม่ตรงทุกเงื่อนไข ให้แสดงข้อความ Error

รับค่าจากผู้ใช้

เรียกใช้ฟังก์ชัน

อธิบายโค้ด

def say_hello(choice):

สร้างฟังก์ชันชื่อ `say_hello()` โดยกำหนด Parameter ชื่อ `choice`

Parameter ตัวนี้ใช้รับค่าที่ส่งเข้ามาจากภายนอก เพื่อใช้ตรวจสอบเงื่อนไขภายในฟังก์ชัน

การแยกออกมาเป็นฟังก์ชันทำให้สามารถเรียกใช้ซ้ำได้หลายครั้ง โดยไม่ต้องเขียนคำสั่ง if ใหม่ทุกครั้ง

if choice == 1:

ตรวจสอบว่าค่าที่รับเข้ามาเป็นเลข 1 หรือไม่

หากเงื่อนไขเป็นจริง โปรแกรมจะทำคำสั่งภายใน if และจะไม่ตรวจสอบเงื่อนไขอื่นต่อ

print("Hello, KMUTNB")

แสดงข้อความทักทายภาษาอังกฤษ

คำสั่งนี้จะทำงานเฉพาะเมื่อ choice มีค่าเท่ากับ 1 เท่านั้น

elif choice == 2:

หากเงื่อนไขแรกไม่เป็นจริง โปรแกรมจะตรวจสอบเงื่อนไขนี้ต่อ

หาก choice มีค่าเป็น 2 โปรแกรมจะแสดงข้อความ "Sawasdee, KMUTNB"

elif choice == 3:

ตรวจสอบว่าผู้ใช้เลือกหมายเลข 3 หรือไม่

หากเป็นจริง จะแสดงข้อความ "Bonjour, KMUTNB"

else:

หากค่าที่ผู้ใช้กรอกไม่ตรงกับทุกเงื่อนไขก่อนหน้า โปรแกรมจะเข้าสู่ส่วน else

ส่วนนี้ช่วยป้องกันการรับค่าที่ไม่ถูกต้อง และแจ้งให้ผู้ใช้ทราบว่าต้องกรอกใหม่

choice = int(input("Enter a number: "))

ใช้คำสั่ง `input()` เพื่อรับข้อมูลจากผู้ใช้

เนื่องจาก `input()` จะคืนค่ากลับมาเป็น String เสมอ จึงต้องใช้ `int()` เพื่อแปลงข้อมูลให้เป็นจำนวนเต็มก่อนนำไปเปรียบเทียบกับตัวเลขในเงื่อนไข

หากไม่แปลงชนิดข้อมูล การเปรียบเทียบ

choice == 1

จะไม่เป็นจริง แม้ว่าผู้ใช้จะพิมพ์เลข 1 ก็ตาม เพราะข้อมูลยังเป็น String

say_hello(choice)

เรียกใช้ฟังก์ชัน พร้อมส่งค่าที่ผู้ใช้กรอกเข้าไปเป็น Argument

เมื่อฟังก์ชันเริ่มทำงาน ค่า Argument จะถูกนำไปเก็บไว้ใน Parameter ชื่อ `choice`

หลังจากนั้นโปรแกรมจึงตรวจสอบเงื่อนไขภายในฟังก์ชัน

Parameter และ Argument

Parameter คือ

choice

ซึ่งถูกประกาศไว้ในฟังก์ชัน

Argument คือค่าที่ส่งเข้าไปจริง เช่น

say_hello(2)

ภายในฟังก์ชัน

choice

จะมีค่าเท่ากับ 2

ทำให้โปรแกรมเข้าสู่เงื่อนไข

elif choice == 2

และแสดงข้อความ "Sawasdee, KMUTNB"

การใช้ Conditional Statement

ข้อนี้เป็นตัวอย่างพื้นฐานของการใช้คำสั่ง if

โปรแกรมจะตรวจสอบเงื่อนไขจากบนลงล่าง

เมื่อพบเงื่อนไขที่เป็นจริงแล้ว โปรแกรมจะข้ามเงื่อนไขที่เหลือทันที

การใช้ if-elif-else ช่วยให้สามารถจัดการหลายกรณีได้อย่างเป็นระเบียบ และเหมาะกับงานที่ต้องเลือกผลลัพธ์เพียงหนึ่งอย่างจากหลายตัวเลือก

Error ที่อาจเกิดขึ้น

หากผู้ใช้กรอกตัวอักษร เช่น

abc

จะเกิด ValueError เพราะคำสั่ง

int()

ไม่สามารถแปลงข้อความให้เป็นจำนวนเต็มได้

หากลืมใช้

int()

เช่น

choice = input(...)

โปรแกรมจะรับข้อมูลเป็น String

ทำให้เงื่อนไข

choice == 1

ไม่เคยเป็นจริง แม้ว่าผู้ใช้จะพิมพ์เลข 1

หากสะกดชื่อฟังก์ชันผิด เช่น

say_helo(choice)

จะเกิด NameError

หากลืมเยื้องภายใน if หรือภายในฟังก์ชัน จะเกิด IndentationError

สิ่งที่ได้เรียนรู้

ข้อนี้ทำให้เข้าใจการทำงานของ **Conditional Statement** และเห็นความแตกต่างระหว่าง **Parameter** และ **Argument** ได้ชัดเจนมากขึ้น นอกจากนี้ยังได้เรียนรู้ว่าข้อมูลที่รับจาก `input()` จะเป็น String เสมอ จึงต้องแปลงชนิดข้อมูลก่อนนำไปใช้งาน การแยกเงื่อนไขทั้งหมดไว้ภายในฟังก์ชันยังช่วยให้โค้ดเป็นระเบียบ อ่านง่าย และสามารถเรียกใช้ซ้ำได้ในอนาคต ซึ่งเป็นหลักการสำคัญของการเขียนโปรแกรมแบบมีโครงสร้าง (Structured Programming)

10. ASCII House

โปรแกรมนี้เป็นการสร้างรูปบ้านด้วยข้อความ (ASCII Art) โดยแบ่งการวาดออกเป็นหลายฟังก์ชัน ได้แก่ `print_roof()` `print_ceiling()` `print_wall()` และ `print_floor()` จากนั้นสร้างฟังก์ชัน `print_house(h)` เพื่อควบคุมการแสดงผลทั้งหมด

จุดประสงค์ของข้อนี้คือการฝึกการแบ่งปัญหาออกเป็นส่วนย่อย (Problem Decomposition) ซึ่งเป็นแนวคิดสำคัญของวิทยาการคอมพิวเตอร์ นอกจากนี้ยังเป็นการประยุกต์ใช้ Function ร่วมกับ Conditional Statement และการรับค่า Parameter เพื่อให้บ้านสามารถมีความสูงแตกต่างกันได้

จุดประสงค์ของข้อนี้

- ฝึกแบ่งโปรแกรมออกเป็นหลายฟังก์ชัน
- ฝึกส่ง Parameter
- ฝึกใช้คำสั่ง if
- ฝึกใช้ Loop ในการทำงานซ้ำ
- ลดการเขียนโค้ดซ้ำ

Pseudocode

สร้างฟังก์ชันวาดหลังคา

สร้างฟังก์ชันวาดเพดาน

สร้างฟังก์ชันวาดผนัง

สร้างฟังก์ชันวาดพื้น

สร้างฟังก์ชัน print_house(h)

แสดงหลังคา

แสดงเพดาน

ทำซ้ำการวาดผนังตามค่า h

แสดงพื้นบ้าน

อธิบายโค้ด

def print_roof()

ใช้สำหรับแสดงหลังคาเพียงครั้งเดียว

เนื่องจากหลังคาไม่มีการเปลี่ยนแปลง จึงไม่จำเป็นต้องรับ Parameter

def print_wall()

ใช้สำหรับแสดงผนังบ้านหนึ่งชั้น

ภายในประกอบด้วยคำสั่ง print() หลายบรรทัด

การแยกออกมาเป็นฟังก์ชันทำให้สามารถเรียกใช้ซ้ำได้โดยไม่ต้องเขียนข้อความเดิมหลายครั้ง

def print_house(h)

เป็นฟังก์ชันหลักของโปรแกรม

Parameter `h` ใช้กำหนดจำนวนชั้นของตัวบ้าน

หากผู้ใช้ส่งค่า 1

จะวาดผนังหนึ่งชุด

หากส่ง 3

จะเรียก `print_wall()` จำนวนสามครั้ง

ทำให้บ้านสูงขึ้นโดยไม่ต้องเขียนโค้ดเพิ่ม

การใช้ Conditional

หากมีการตรวจสอบ

if h <= 0

โปรแกรมสามารถป้องกันค่าที่ไม่ถูกต้องได้

เช่น หากผู้ใช้กรอกเลขติดลบ โปรแกรมสามารถแจ้งเตือนและไม่สร้างบ้าน

การใช้ if ทำให้โปรแกรมมีความน่าเชื่อถือมากขึ้น

Error ที่อาจเกิดขึ้น

หากลืมส่งค่า h

จะเกิด TypeError เพราะฟังก์ชันต้องการ Parameter

หากส่ง String แทน Integer

อาจเกิด TypeError เมื่อใช้ร่วมกับ range()

หากใช้ Recursive Function โดยไม่มีเงื่อนไขหยุด

อาจเกิด RecursionError ได้

หากลืมเยื้องภายใน if หรือ for จะเกิด IndentationError

สิ่งที่ได้เรียนรู้

ข้อนี้เป็นการสรุปความรู้ทั้งหมดของ Lab 2 ไม่ว่าจะเป็น Function, Parameter, Loop และ Conditional เข้าด้วยกัน การแบ่งบ้านออกเป็นหลายฟังก์ชันช่วยให้โค้ดอ่านง่าย แก้ไขง่าย และสามารถเพิ่มความสูงของบ้านได้เพียงเปลี่ยนค่า Parameter โดยไม่ต้องแก้ไขคำสั่งภายในฟังก์ชัน ถือเป็นตัวอย่างที่ชัดเจนของการออกแบบโปรแกรมที่ดีและสามารถต่อยอดไปสู่โปรแกรมที่มีขนาดใหญ่ขึ้นได้ในอนาคต

11. Password Analyzer

โปรแกรมนี้เป็นการสร้างโปรแกรมตรวจสอบความแข็งแรงของรหัสผ่าน (Password Analyzer) โดยใช้ **User-defined Function** ร่วมกับ **Conditional Statement** เพื่อพิจารณาว่ารหัสผ่านที่ผู้ใช้ป้อนมีคุณสมบัติตรงตามเงื่อนไขที่กำหนดหรือไม่ โปรแกรมจะตรวจสอบว่ารหัสผ่านมีความยาวอย่างน้อย 8 ตัวอักษร มีตัวอักษรพิมพ์ใหญ่ ตัวอักษรพิมพ์เล็ก ตัวเลข และอักขระพิเศษ (Special Character) อย่างน้อยอย่างละหนึ่งตัว หากขาดคุณสมบัติข้อใด โปรแกรมจะแจ้งให้ผู้ใช้ทราบทันที แต่หากครบทุกเงื่อนไข โปรแกรมจะแสดงข้อความว่า Password Accepted

จุดประสงค์ของข้อนี้คือการฝึกแบ่งโปรแกรมออกเป็นหลายฟังก์ชัน การเรียกใช้ฟังก์ชันซ้ำ การใช้คำสั่งเงื่อนไข และการประยุกต์ใช้ฟังก์ชัน `ord()` เพื่อตรวจสอบรหัส ASCII ของตัวอักษร ซึ่งเป็นอีกวิธีหนึ่งที่สามารถใช้จำแนกประเภทของตัวอักษรได้โดยไม่ต้องใช้ฟังก์ชันสำเร็จรูปของภาษา Python

จุดประสงค์ของข้อนี้

- ฝึกสร้าง User-defined Function หลายฟังก์ชัน
- ฝึกเรียกใช้ฟังก์ชันร่วมกัน
- ฝึกใช้ Parameter และ Return Value
- ฝึกใช้ Conditional Statement
- ฝึกใช้ฟังก์ชัน `ord()` เพื่อเปรียบเทียบรหัส ASCII
- ฝึกออกแบบโปรแกรมโดยแบ่งหน้าที่ของแต่ละฟังก์ชันให้ชัดเจน

Pseudocode

รับรหัสผ่านจากผู้ใช้

ตรวจสอบความยาวของรหัสผ่าน

ตรวจสอบตัวอักษรพิมพ์ใหญ่

ตรวจสอบตัวอักษรพิมพ์เล็ก

ตรวจสอบตัวเลข

ตรวจสอบอักขระพิเศษ

หากผ่านทุกเงื่อนไข แสดง Password Accepted

อธิบายโค้ด

```
def check_ascii(password, start, end):
```

สร้างฟังก์ชันสำหรับตรวจสอบว่ารหัสผ่านมีตัวอักษรที่มีค่า ASCII อยู่ในช่วงที่กำหนดหรือไม่ ฟังก์ชันนี้รับ Parameter ทั้งหมด 3 ตัว ได้แก่ รหัสผ่าน จุดเริ่มต้นของช่วง ASCII และจุดสิ้นสุดของช่วง ASCII

ทำให้สามารถนำฟังก์ชันเดียวกันไปใช้ตรวจสอบตัวอักษรหลายประเภทได้ เช่น ตัวพิมพ์ใหญ่ ตัวพิมพ์เล็ก และตัวเลข โดยไม่ต้องเขียนโค้ดซ้ำ

```
for character in password:
```

วนลูปตรวจสอบตัวอักษรทุกตัวในรหัสผ่านทีละตัว เพื่อให้สามารถนำค่า ASCII ของแต่ละตัวมาเปรียบเทียบกับช่วงที่ต้องการได้

```
ord(character)
```

ฟังก์ชัน `ord()` ใช้แปลงตัวอักษรให้เป็นรหัส ASCII เช่น

A = 65

Z = 90

a = 97

z = 122

0 = 48

9 = 57

เมื่อได้ค่า ASCII แล้ว โปรแกรมสามารถเปรียบเทียบเป็นช่วงตัวเลขได้ทันที ทำให้ตรวจสอบประเภทของตัวอักษรได้อย่างมีประสิทธิภาพ

```
return True
```

หากพบตัวอักษรที่อยู่ในช่วงที่กำหนด ฟังก์ชันจะคืนค่า `True` ทันที และหยุดการทำงานของฟังก์ชัน ทำให้ไม่ต้องตรวจสอบตัวอักษรที่เหลือต่อ ซึ่งช่วยลดเวลาการประมวลผล

```
return False
```

หากตรวจสอบครบทุกตัวแล้วไม่พบตัวอักษรที่ตรงตามเงื่อนไข ฟังก์ชันจะคืนค่า `False` เพื่อแจ้งให้ฟังก์ชันหลักทราบว่ารหัสผ่านยังไม่ผ่านเงื่อนไขนั้น

```
def check_special(password):
```

สร้างฟังก์ชันสำหรับตรวจสอบอักขระพิเศษ (Special Character) โดยไม่ใช้คำสั่งสำเร็จรูปของ Python แต่ใช้การเปรียบเทียบช่วงค่า ASCII แทน ซึ่งทำให้เข้าใจโครงสร้างของชุดอักขระ ASCII มากขึ้น

ภายในฟังก์ชันจะตรวจสอบช่วง ASCII ดังนี้

33–47

58–64

91–96

123–126

ซึ่งครอบคลุมอักขระพิเศษ เช่น

!

@

$

%

^

&

-

(

)

_

-

และเครื่องหมายอื่น ๆ อีกหลายชนิด

หากพบอักขระที่อยู่ในช่วงเหล่านี้ ฟังก์ชันจะคืนค่า `True` ทันที

```
def password_checker(password):
```

เป็นฟังก์ชันหลักของโปรแกรม มีหน้าที่เรียกใช้ฟังก์ชันย่อยทั้งหมดเพื่อตรวจสอบรหัสผ่านทีละเงื่อนไข

```
if len(password) < 8:
```

ตรวจสอบความยาวของรหัสผ่านก่อนเป็นอันดับแรก หากมีความยาวน้อยกว่า 8 ตัวอักษร โปรแกรมจะแจ้งเตือนทันที เพราะความยาวเป็นเงื่อนไขพื้นฐานที่สุดของรหัสผ่านที่ปลอดภัย

```
elif not check_ascii(password, 65, 90):
```

เรียกใช้ฟังก์ชัน `check_ascii()` เพื่อตรวจสอบว่ามีตัวอักษรภาษาอังกฤษพิมพ์ใหญ่อย่างน้อยหนึ่งตัวหรือไม่ โดยช่วง ASCII 65–90 คือ A ถึง Z

```
elif not check_ascii(password, 97, 122):
```

ตรวจสอบตัวอักษรพิมพ์เล็ก โดยใช้ช่วง ASCII 97–122 ซึ่งตรงกับ a ถึง z

`elif not check_ascii(password, 48, 57):`

ตรวจสอบว่ารหัสผ่านมีตัวเลขหรือไม่ โดยใช้ช่วง ASCII 48–57 ซึ่งตรงกับเลข 0–9

`elif not check_special(password):`

เรียกใช้ฟังก์ชัน `check_special()` เพื่อตรวจสอบว่ามีอักขระพิเศษอย่างน้อยหนึ่งตัวหรือไม่

`print("Password Accepted")`

หากรหัสผ่านผ่านทุกเงื่อนไข โปรแกรมจะแสดงข้อความยืนยันว่ารหัสผ่านสามารถใช้งานได้

`password = input("Enter password:")`

รับรหัสผ่านจากผู้ใช้ผ่านทางคีย์บอร์ด และเก็บไว้ในตัวแปร `password`

`password_checker(password)`

เรียกใช้ฟังก์ชันหลัก พร้อมส่งรหัสผ่านที่ผู้ใช้ป้อนเข้าไปเป็น Argument เพื่อเริ่มกระบวนการตรวจสอบทั้งหมด

Parameter และ Return Value

โปรแกรมนี้ใช้ Parameter หลายตัว เช่น `password`, `start` และ `end` เพื่อทำให้ฟังก์ชันสามารถนำกลับมาใช้ซ้ำได้กับหลายกรณี ส่วนการใช้ `return True` และ `return False` ช่วยให้ฟังก์ชันสามารถส่งผลลัพธ์กลับไปยังฟังก์ชันหลักได้ ทำให้การออกแบบโปรแกรมมีความยืดหยุ่นและลดการเขียนโค้ดซ้ำ

Error ที่อาจเกิดขึ้น

หากลืมใส่ Parameter ขณะเรียกใช้ `check_ascii()` จะเกิด **TypeError** เนื่องจากจำนวน Argument ไม่ครบตามที่ฟังก์ชันต้องการ

หากสะกดชื่อฟังก์ชันผิด เช่น `check_ascci()` จะเกิด **NameError** เพราะ Python ไม่พบฟังก์ชันดังกล่าว

หากลืม `return False` เมื่อไม่พบตัวอักษรที่ตรงตามเงื่อนไข ฟังก์ชันจะคืนค่า `None` แทน ทำให้เงื่อนไขภายใน `password_checker()` ทำงานผิดจากที่คาดไว้

หากเยื้องบรรทัดภายในฟังก์ชันไม่ถูกต้อง จะเกิด **IndentationError**

หากใช้ช่วง ASCII ผิด เช่น ตรวจสอบตัวพิมพ์ใหญ่ด้วยช่วง 60–90 โปรแกรมอาจยอมรับอักขระที่ไม่ใช่ตัวอักษร ทำให้การตรวจสอบรหัสผ่านไม่ถูกต้อง ซึ่งถือเป็น **Logical Error**

สิ่งที่ได้เรียนรู้

ข้อนี้ทำให้เข้าใจการออกแบบโปรแกรมโดยแบ่งหน้าที่ของแต่ละฟังก์ชันอย่างชัดเจน แต่ละฟังก์ชันรับผิดชอบเพียงงานเดียว ทำให้สามารถนำกลับมาใช้ซ้ำได้ในหลายสถานการณ์ นอกจากนี้ยังได้เรียนรู้

การใช้ `return` เพื่อส่งค่ากลับจากฟังก์ชัน การใช้ `ord()` เพื่อเปรียบเทียบรหัส ASCII และการใช้ `if-elif-else` เพื่อควบคุมลำดับการตรวจสอบเงื่อนไขอย่างเป็นระบบ ซึ่งเป็นแนวคิดสำคัญของการเขียนโปรแกรมเชิงโครงสร้างและสามารถนำไปประยุกต์ใช้กับงานจริงได้ในอนาคต